



Universidad Autónoma del Estado de México

Diplomado Superior en Técnicas y Herramientas
Computacionales para el Análisis de Datos

Ingeniería en Computación (ICO)

Redes Neuronales

Alumno: Trejo Amador Axel

Instructor: M. en C. Yeredith Giovanna Mora Martínez

Actividad 1: Implementación de una Red Neuronal Simple

Fecha: 11 / 08 / 2026



ÍNDICE

Introducción	2.
Desarrollo	3.
Cuestionario	10.
Conclusión	13.



INTRODUCCIÓN.

Durante el avance del análisis de datos como una ciencia y una metodología para la investigación en diversos campos llegamos al área cúspide de este tema siendo la inteligencia artificial y con este termino llega de la mano con el término “Redes Neuronales Artificiales”.

Siendo estas una representación de la innovación y el crecimiento tecnológico a nivel conocimiento de la capacidad de la programación, análisis y base de datos para facilitar y automatizar respuestas a problemas cotidianos a cualquier nivel y ámbito tanto doméstico, escolar, profesional, científico, comercia y político, teniendo como visión la adaptación de IAS en la vida de cada persona.

Con esta percepción lo que se plantea realizar en estas clases es la introducción conceptual y desarrollo de pequeñas redes neuronales con las cuales podremos ver e interpretar el gran auge que ha tenido la IA como la estructura e infraestructura de cómo funcionan más allá de realizar preguntas para poder satisfacer las problemáticas cotidianas. Es ir mas adentro de su funcionamiento y poder recrear una parte de las redes neuronales.



DESARROLLO.

Código fuente utilizado.

```
if __name__ == '__main__':

    """
    Accion:
        Inicializamos las variables y mandamos a llamar las funciones para el calculo de
        "pesos_finales", "sesgo_final" y "errores_por_epoca" y poder graficar el resultado
        de la neurona.
    """

    #np.array sirve para crear arreglos multidimensionales
    x = np.array([ [0, 0], [0, 1], [1, 0], [1, 1] ])
    y = np.array([0, 0, 0, 1])
    tasa_aprendizaje = 0.1
    epocas = 10

    pesos_finales, sesgo_final, errores_por_epoca = entrenar_perceptron(x, y, tasa_aprendizaje, epocas)

    print("\n")
    print("Pesos finales:", pesos_finales)
    print("Sesgo final:", sesgo_final)
    print("\n")

    print("Pruebas del modelo entrenado:")
    for entrada in x:
        salida = predecir(entrada, pesos_finales, sesgo_final)
        print("Entrada:", entrada, "Predicción:", salida)

    graficacion(errores_por_epoca, epocas)

    print("-----")
    entrada_1 = np.array([0, 0])
    entrada_2 = np.array([0, 1])
    entrada_3 = np.array([1, 0])
    entrada_4 = np.array([1, 1])
    print("Entrada [0, 0]:", predecir(entrada_1, pesos_finales, sesgo_final))
    print("Entrada [0, 1]:", predecir(entrada_2, pesos_finales, sesgo_final))
    print("Entrada [1, 0]:", predecir(entrada_3, pesos_finales, sesgo_final))
    print("Entrada [1, 1]:", predecir(entrada_4, pesos_finales, sesgo_final))
```



```

def entrenar_perceptron(X, y, tasa_aprendizaje, epocas):
    """
        variables "parametros":
            X: arreglo multidimensional
            y: arreglo multidimensional
            tasa_aprendizaje: valor doble
            epocas: valor entero

        variables:
            error_total: valor cero
            pesos: arreglo multidimensional en ceros
            errores_por_epocas: arreglo vacio
            sesgo: valor entero
            prediccion: valor retornado de la funcion "predecir"

        accion:
            Realiza un ciclo anidado del rango de "epocas" y lonitud
            de "x" donde en este ultimo ciclo obtendremos valores
            redefinidos de variables "prediccion", "error", "pesos",
            "sesgo" y "error_total", mientras al finalizar cada ciclo
            de "epocas" se almacenara "error_total" en el arreglo
            "errores_por_epocas"

        return:
            pesos
            sesgo
            errores_por_epoca
    """

    # no.zeros crea un arreglo de la misma caracteristica del arreglo multidimensional
    # X pero este nuevo arreglo esta lleno de ceros y el shape[#], indicara cuantas
    # columnas va a tomar
    pesos = np.zeros(X.shape[1])
    sesgo = 0.0
    errores_por_epoca = []
    for epoca in range(epocas):
        error_total = 0
        for i in range(len(X)):
            prediccion = predecir(X[i], pesos, sesgo)
            error = y[i] - prediccion
            pesos = pesos + tasa_aprendizaje * error * X[i]
            sesgo = sesgo + tasa_aprendizaje * error
            error_total = error_total + abs(error)
        errores_por_epoca.append(error_total)
        print("Época:", epoca + 1)
        print("Error total:", error_total)
        print("Pesos:", pesos)
        print("Sesgo:", sesgo)
        print("-----")

    return pesos, sesgo, errores_por_epoca

```



```

def predecir(entradas, pesos, sesgo):
    """
        variables "parametros":
            entradas: valor indice de un arreglo multidimensional
            pesos: arreglo multidimensional
            sesgo: valor

        variables:
            suma_ponderada: valor entero
            salida: valor entero

        accion:
            Realiza una sumatoria con no.dot() viendose asi:
            |entrada| * |pesos|
            [0, 0] · [0., 0.]

            Nota: Entrada puede cambiar de [0,0] a [1,1] eso
            dependera de como hayas hecho el arreglo y que indice
            le pases como entrada.

            una vez teniendo la "suma_ponderada" la pasamos como parametro
            a "funcion_vectorial" y este retornara una salida

        return:
            salida

    """
    suma_ponderada = np.dot(entradas, pesos) + sesgo
    salida = funcion_activacion(suma_ponderada)
    return salida

```



```

def funcion_activacion(valor):
    """
        variables "parametros"
        |
        | valor: valor double

        accion:
        |
        | compara si el valor es mayor a 0 ó menor a 1

        return:
        |
        | 0 ó 1
    """

    if valor >= 0:
        |
        | return 1
    else:
        |
        | return 0

def graficacionerrores_por_epoca,epocas):
    """
        variables "parametros":
        |
        | errores_por_epoca: valor entero
        |
        | epocas: valor entero

        accion:
        |
        | Graficamos para el eje "Y" = "errores por epocas"
        |
        | mientras el eje "X" = "epocas"

    """

    print(f'{errores_por_epoca}')
    print(f'{epocas}')
    plt.plot(range(1, epocas + 1), errores_por_epoca, marker='*')
    plt.title("Error durante el entrenamiento")
    plt.xlabel("Época")
    plt.ylabel("Error total")
    plt.grid(True)
    plt.show()

```



Capturas de pantalla de la ejecución.

- Captura de Funcion: `entrenar_perceptron(x, y, tasa_aprendizaje, epocas)`

<pre>Época: 1 Error total: 2 Pesos: [0.1 0.1] Sesgo: 0.0 ----- Época: 2 Error total: 3 Pesos: [0.2 0.1] Sesgo: -0.1 ----- Época: 3 Error total: 3 Pesos: [0.2 0.1] Sesgo: -0.20000000000000004 ----- Época: 4 Error total: 0 Pesos: [0.2 0.1] Sesgo: -0.20000000000000004 ----- Época: 5 Error total: 0 Pesos: [0.2 0.1] Sesgo: -0.20000000000000004 -----</pre>	<pre>Época: 6 Error total: 0 Pesos: [0.2 0.1] Sesgo: -0.20000000000000004 ----- Época: 7 Error total: 0 Pesos: [0.2 0.1] Sesgo: -0.20000000000000004 ----- Época: 8 Error total: 0 Pesos: [0.2 0.1] Sesgo: -0.20000000000000004 ----- Época: 9 Error total: 0 Pesos: [0.2 0.1] Sesgo: -0.20000000000000004 ----- Época: 10 Error total: 0 Pesos: [0.2 0.1] Sesgo: -0.20000000000000004 -----</pre>
--	--

Pesos Finales y Sesgos Finales.

- Captura de los Pesos Finales y Sesgo Final:

```
Pesos finales: [0.2 0.1]
Sesgo final: -0.20000000000000004
```



Predicciones del Modelo.

- Captura de la prueba del modelo.

```
Pruebas del modelo entrenado:  
Entrada: [0 0] Predicción: 0  
Entrada: [0 1] Predicción: 0  
Entrada: [1 0] Predicción: 0  
Entrada: [1 1] Predicción: 1
```

Tabla de verdad de la compuerta AND y

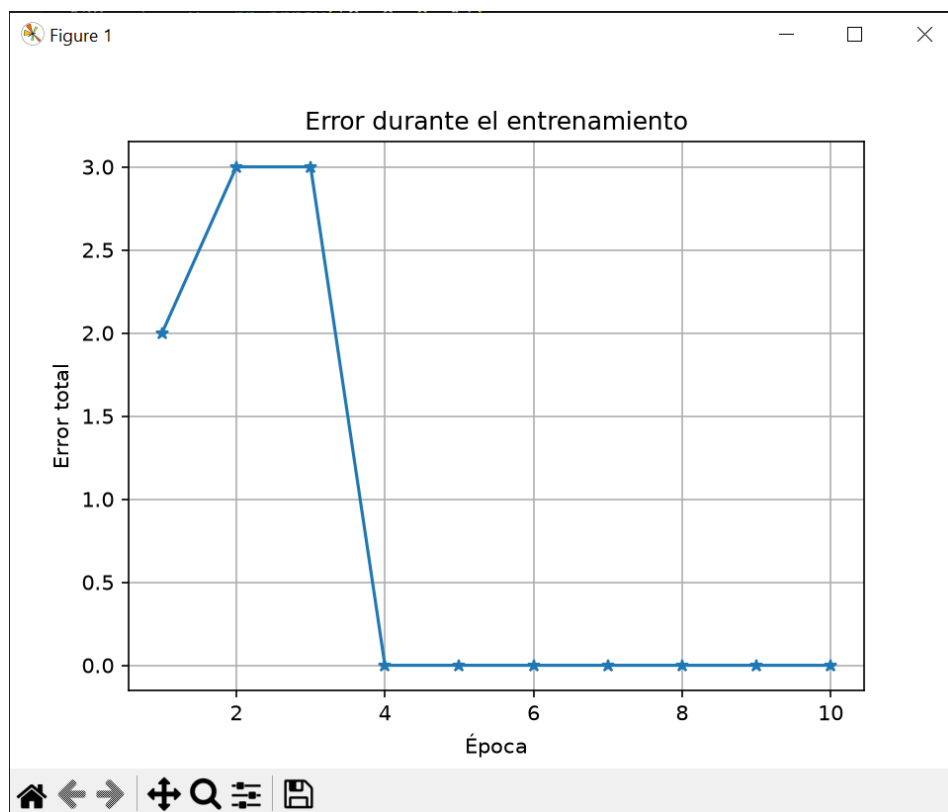
- Captura de pruebas independientes a la Funcion:
“entrenar_perceptron()”

```
Pruebas independientes al proceso 'Pruebas'.  
Entrada [0, 0]: 0  
Entrada [0, 1]: 0  
Entrada [1, 0]: 0  
Entrada [1, 1]: 1  
(.venv13) PS C:\Users\pumas\OneDrive\Documents\Portafolio-Diplomado>
```



Gráfica del error durante el entrenamiento.

Captura de la grafica obtenida de los resultados de Funcion:
“entrenar_perceptron()”



CUESTIONARIO

Respuestas de la actividad de integración.

1. ¿Qué es una neurona artificial?

R: Son modelados simulando el funcionamiento del cerebro humano

2. ¿Qué representan las entradas en el perceptrón?

R: 'X' son las entradas del modelo

'y' es salida

'taza_aprendizaje' es aquel que controla el tamaño y ajusta durante el entrenamiento

'época' es indica el número de veces que se ejecutara el entrenamiento

3. ¿Qué función cumplen los pesos dentro del modelo? •

R: mide la importancia de la columna o característica para el resultado

4. ¿Qué es el sesgo y por qué es importante? •

R: es el error, es importante ya que indica al perceptrón aprenda poco a poco de sus error

5. ¿Qué hace la función de activación escalón? •

R: Retorna un valor 1 o 0

6. ¿Cómo se calcula el error del modelo? •

R: $\text{error} = \text{salida_esperada} - \text{prediccion}$

7. ¿Qué ocurre cuando el error es igual a cero? •

R: La predicción es correcta

8. ¿Qué ocurre cuando la predicción es diferente a la salida esperada? •

R: La salida es correcta

9. ¿Por qué se actualizan los pesos durante el entrenamiento? •

R: para ajustar la importancia de cada entrada

10. ¿Qué indica la gráfica del error? •

R: Como avanza la época con respecto al error



11. ¿El perceptrón logró aprender la compuerta AND?

Justifica tu respuesta.

R: SI. porque durante el proceder de las épocas el perceptrón va aprendiendo el comportamiento y va ajustando los valores para llegar al resultado una vez cumplido el ciclo.



CONCLUSION.

La conclusión a la que llego es que puede ser algo confuso al momento de llegar a realizar la programación, ya que habitualmente la lógica y la matemática se percibía únicamente a datos e interpretación escrita, ya pasarla a código puede ser algo confuso, el tener una base lógica de como funciona tanto la lógica de programación, como las matrices, las compuertas y una vez aislada la terminología es mucho más sencillo.

Ahora hablando un poco mas respecto a la clase, la implementación de la primera neurona y la vista grafica obtenida es retador, pero a su vez nos muestra una percepción de un mundo para la fácil lectura interpretación y análisis de los datos que queramos analizar en un futuro.

